The Java Class Libraries

Why we use libraries

Java Class Libraries

- Also known as the Java API,provide a vast colle ction of prewritten classes and interfaces that simplify Java programming for beginners.
- Library is a comprehensive set of reusable classes, interfaces, and APIs that come bundled with the Java Development Kit (JDK).

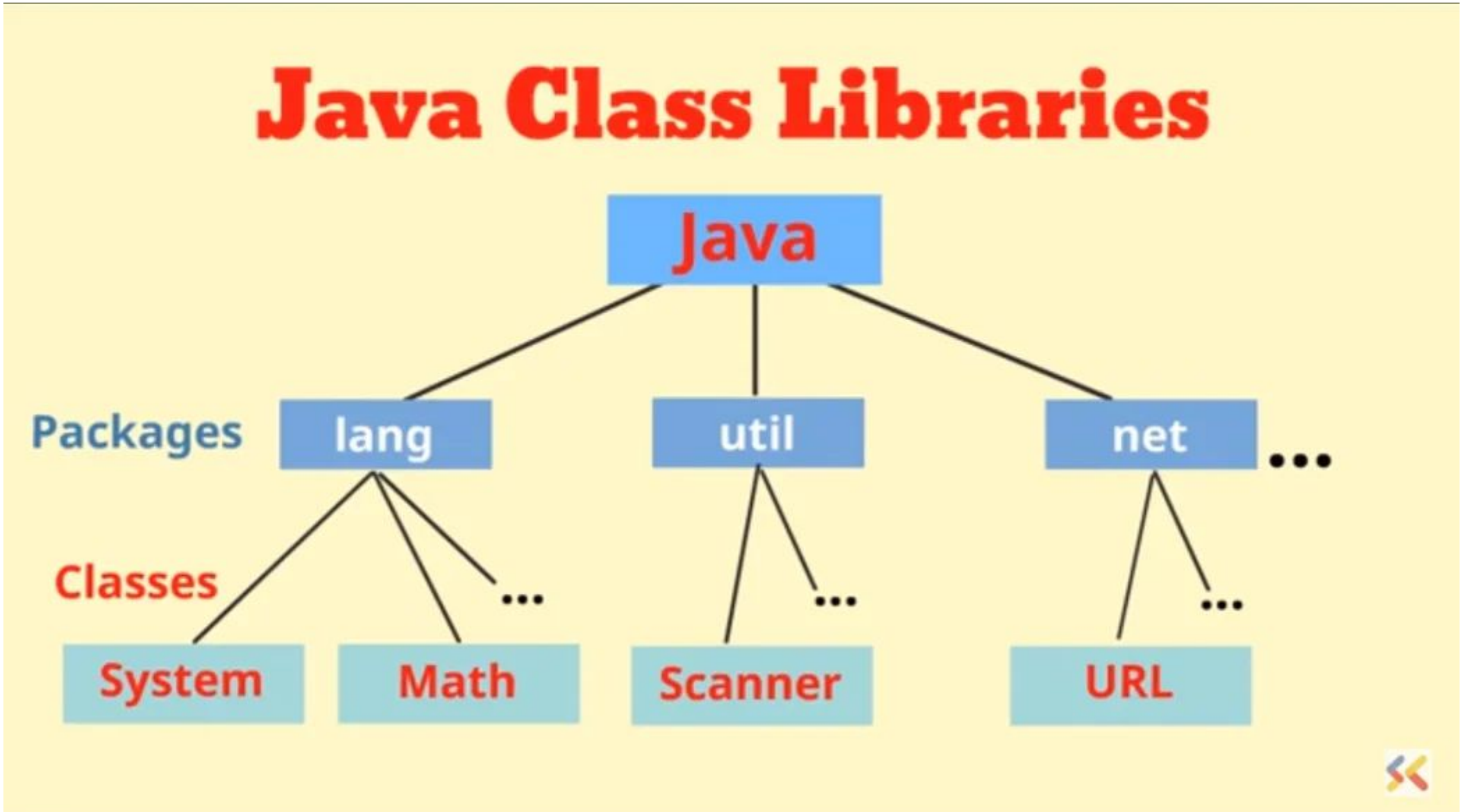
- Java is not dependent on any specific operating system so Java applications cannot depend on the platform dependent native libraries, therefore, instead of those Java provides a set of dynamically loaded libraries that are common to modern operating systems.

- Java libraries save development time by providing pre-tested, reusable components.
- They eliminate redundancy and enhance productivity, making them essential for modern Java development.

libraries Provides

- Container classes and Regular Expressions.
- Interfaces for tasks that depend on the hardware of the OS such as network and file access.
- In case, the underlying platform does not support certain feature of Java then, these libraries surpass that specific feature if needed.

- Java Class Library (JCL) are dynamically loadable libraries that Java applications can call at run time. Some examples of JCL are java.lang, java.util, java.math, java.awt. etc.



Java Class Libraries
Java
Packages
lang
util
net
...
Classes
System
Math
...
Scanner
...
URL
...

- Java contains an extensive library of pre-written classes (JCL) you can use in your programs.
- These classes are divided into groups called packages.
- A Package is a collection of related classes, providing access protection and name place management.

Java Package

- Java uses packages because
  - It makes search and re-using packages and classes easier.
  - It prevents naming conflicts. You can have a class with same name in two different package.
  - Provides controlled access as you can specify protected and default access specifier for methods which work within a package

Java Standard Library

Java provides a large number of classes groped into different packages based on their functionality

- The six foundation Java packages are:
  - java.lang :

    Contains classes for primitive types, strings, math functions, threads, and exception.
  - java.util :

    Contains classes such as vectors, hash tables, date etc.
  - java.io :

    Stream classes for I/O .
  - java.awt :

    Classes for implementing GUI – windows, buttons, menus etc.
  - java.net :

    Classes for networking .
  - java.applet :

    Classes for creating and implementing app.

Core Utility Libraries

- **Apache Commons**: A collection of reusable Java compo nents for tasks like string manipulation
  - (commons-lang),
  - collections (commons-collections),
  - IO (commons-io),
  - math (commons-math), and more.
- **Guava (Google Core Libraries)**: Provides utilities for
  - collections, caching, primitives, concurrency, IO, and string manipulation.
  - It is known for its immutable collections and functional-style utilities.

Logging Libraries

- **SLF4J (Simple Logging Facade for Java)**:
  - A facade for various logging frameworks, allowing abstraction over the logging implementation.
- **Logback**:
  - A successor to Log4j, used with SLF4J for high-perf ormance logging.
- **Log4j 2**:
  - Widely used flexible and performance logging libr ary with asynchronous logging support.

Data and Database Libraries

- **Hibernate ORM**:
  - A popular object relational mapping (ORM) framework for mapping Java objects to relational databases, with advanced query capabilities (HQL).
- **JDBC and Apache DbUtils**:
  - JDBC is part of JDK while DbUtils simplify database access and avoid boilerplate code.
- **MyBatis**:
  - A SQL mapper framework which allows fine-grained SQL control along with object mapping.

Web and Networking Libraries

- **Spring Framework**:
  - Provides comprehensive infrastructure support for d eveloping Java applications. It contains modules for Spring MVC, Spring Boot, and Spring Data.
- **Apache HttpClient**:
  - Provides HTTP client APIs for sending HTTP requests and handling responses.
- **OkHttp**:
  - An efficient HTTP client for Java and Android.

JSON and XML Processing

- **Jackson**:
  - Highperformance JSON processor for converting Java obj ects to/from JSON.
- **Gson (Google)**:
  - Lightweight library for JSON parsing and serialization.
- **JAXB**:
  - Java Architecture for XML Binding for converting Java ob jects to XML and vice versa.

Testing Libraries

- **JUnit**: Core framework for unit testing Java code.
- **TestNG**:
  - Feature-rich testing framework geared for testing with configuration flexibility and parallel execution.
- **Mockito**: Popular mocking framework for creating test doubles in unit tests.

Concurrency & Utilities

- **java.util.concurrent**:
  - Part of JDK; provides thread pools, executors, and concurrent data structures.
- **Akka**:
  - Toolkit and runtime for building highly concurrent, distributed, and resilient message driven applications.

Build and Dependency Management

- Although technically not libraries, tools like

Maven and **Gradle** are essential in Java projects for managing libraries and builds efficiently.